

RAMA: INFORMÁTICA	CICLO: DESENVOLVIMIENTO DE APLICACIONES MULTIPLATAFORMA
MÓDULO: ACCESO A DATOS	CURSO: 2º
PROFESOR: IAGO DOVAL IGLESIAS	
AVALIACIÓN: RECUPERACIÓN	GRUPO: ÚNICO DATA: Junio 2026
NOMBRE ALUMNO:	DNI:

El examen consta de **tres partes**. Para aprobar hay que obtener un mínimo de **5 puntos** en total, con al menos **1 punto** en cada parte. Sube un único fichero comprimido con todos los archivos utilizados con el formato: **1erApellido_2oApellido_Nombre**.

En todos los ejercicios: trata las excepciones adecuadamente y libera todos los recursos (-0.10 pt por elemento no tratado). Muestra un ejemplo de invocación en cada ejercicio (-0.15 pt si no se indica).

PARTE 1 – Ficheros (3 pt)

- 1) **Ficheros (1.30 pt)**. Crea el método **reemplaza** que, dado dos ficheros y dos cadenas, sustituya todas las apariciones de la primera cadena por la segunda en el primer fichero. El resultado se almacenará en el segundo fichero. No se pueden usar estructuras en memoria (arrays, ArrayList, HashMap...) y no hay que realizar lecturas a disco innecesarias.
- 2) **Ficheros (1.70 pt)**. Crea el método **transform** que transforme un fichero binario que almacena tipos primitivos a un fichero de texto. El fichero binario contiene secuencias de *Int String Int String ...* donde el entero indica el número de veces que se escribirá la cadena en el fichero de texto de salida. Como datos de prueba puedes usar **transform.dat**.

PARTE 2 – DOM / SAX (3.70 pt)

El fichero **Alonso.xml** contiene información sobre 5 carreras de F1 disputadas por el piloto. Toda la información mostrada debe extraerse de este fichero.

- 3) **DOM (0.90 pt)**. Crea el método **pacesD** que muestre de cada carrera (Race): el nombre de la carrera (Name dentro de Race), su latitud y longitud (atributos lat y long de Location), el nombre del circuito (Name dentro de Circuit) y la posición del piloto (atributo position de Result).
- 4) **DOM (1.00 pt)**. Crea el método **velocidadD** que muestre la velocidad media (AverageSpeed) más alta alcanzada por Alonso, indicando el nombre de la carrera (Name dentro de Race) en la que se consiguió.
- 5) **SAX (0.70 pt)**. Crea el manejador **Nombres** que muestre, en este orden, el apellido (Surname) del piloto, su número (Number) y el nombre de la escudería (Name dentro de Constructor).
- 6) **SAX (1.10 pt)**. Crea el manejador **RacesS** que muestre, para cada carrera (Race), el nombre del circuito (Name dentro de Circuit), su latitud y longitud (atributos lat y long de Location) y la posición del piloto (atributo position de Result). Además, muestra también el código del circuito (atributo code de Circuit).

PARTE 3 – JSON (3.30 pt)

El fichero **Sainz.json** contiene información sobre 5 carreras de F1 disputadas por el piloto. Toda la información mostrada debe extraerse de este fichero.

- 7) **JSON (0.70 pt)**. Crea el método **nombreJ** que muestre, en este orden, el nombre (Name) del piloto, su número (Number) y el nombre de la escudería (Name dentro de Constructor).
- 8) **JSON (1.30 pt)**. Crea el método **pacesJ** que muestre de cada carrera (Race) el nombre de la carrera (Name en Races), su latitud y longitud (lat y long de Location), el nombre del circuito (Name dentro de Circuit) y la posición en la carrera (position en Results).
- 9) **JSON (1.30 pt)**. Crea el método **finalizadasJ** que muestre el nombre de las carreras que ha terminado (status con valor *Finished*). Además, para cada carrera finalizada, muestra también la velocidad media (speed) obtenida.

